

Spark Declarative Pipelines

Hands-On Lab — CLI Deployment with Databricks Asset Bundles

Overview

This lab walks you through deploying the SDP pipeline you wrote in Lab 1 using a Databricks Asset Bundle (DAB) from your terminal. You will scaffold a new bundle with the default Python template, swap the sample Python transformations for the four SQL files from Lab 1, then validate, preview, deploy, and run the pipeline — all from the CLI.

Prerequisites

- Databricks CLI v0.220+ installed (check with `databricks --version`).
- Network access to your target workspace.
- The Lab 1 bootstrap notebook ready to run. You will choose a catalog and schema during the workshop — pick once, then use the same values everywhere (bootstrap widgets, bundle variables, SQL FROM clauses).

One-time shell setup

Set the bundle engine to direct mode before you start. The default engine ships with a bundled Terraform provider key that has expired in some environments; direct mode bypasses it and is otherwise equivalent. Add this to your shell once per terminal session:

```
export DATABRICKS_BUNDLE_ENGINE=direct
```

Part 1 — Authenticate the CLI

You should already have the Databricks CLI configured. Verify; if it fails, run an OAuth login against your target workspace.

#	Action	Details
1	Verify current auth	<code>databricks current-user me</code> If this returns your user JSON, skip to Part 2.
2	If not authenticated, start an OAuth login	<code>databricks auth login --host https://<your-workspace>.cloud.databricks.com</code> The CLI opens a browser tab. Sign in if prompted and approve the CLI. The CLI prints "Profile <name> was successfully saved".
3	Re-verify	<code>databricks current-user me</code> If you saved a non-default profile, append <code>--profile <name></code> .

Part 2 — Scaffold the Bundle

#	Action	Details
1	Move to your projects directory	<pre>cd <your projects dir></pre> The bundle becomes a new folder under whatever directory you're in. Pick a location you can find again — e.g., your usual git workspace.
2	Run the initializer	<pre>databricks bundle init</pre> Choose the default Python template for Declarative Automation Bundles when prompted.
3	Answer the prompts	Use the following answers (substitute your own naming for the project and catalog if your environment differs): <pre>Unique name for this project: ball_test Include a job that runs a notebook: yes Include an ETL pipeline: yes Include a sample Python package that builds into a wheel: no Use serverless compute: yes Default catalog for any tables created by this project: demos Use a personal schema for each user working on this project: yes Pick the same catalog you'll use for the Lab 1 bootstrap. The 'personal schema' answer wires the bundle to use \$ {workspace.current_user.short_na me} as the schema in dev (e.g., ethan_feldman).</pre>
4	Inspect what you got	<pre>cd ball_test && ls</pre> You should see databricks.yml, resources/, src/ball_test_etl/transformations/ (with two sample .py files), pyproject.toml, README.md, and tests/. Open databricks.yml to confirm the catalog/schema variables match what you answered.

Part 3 — Replace the Sample Python with SDP SQL

The template ships with two sample Python transformations (sample_trips_ball_test.py and sample_zones_ball_test.py) that read from samples.nyctaxi. Delete them and drop in the four SQL files from Lab 1 instead. The pipeline.yml glob already matches everything under transformations/**, so .sql files are picked up automatically — no YAML changes needed.

#	Action	Details
1	Run the Lab 1 bootstrap first	Open and run 00_bootstrap_bronze.py from Lab 1. Set its catalog and schema widgets to the values you'll use for this bundle (e.g., catalog=demos, schema=<your personal schema>). Remember these values — you will reference them again in the silver SQL FROM clauses and in databricks.yml.
2	Delete the sample Python transformations	<pre>rm src/ball_test_etl/transformations/sample_trips_ball_test.py rm src/ball_test_etl/transformations/sample_zones_ball_test.py</pre>
3	Create silver_customers.sql	Create src/ball_test_etl/transformations/silver_customers.sql and paste the silver_customers query from the Lab 1 Query Reference. Update the FROM clause to your bronze location: <pre>FROM <catalog>.<schema>.dim_customers_raw</pre>
4	Create silver_orders.sql	Create silver_orders.sql with the silver_orders query. Update the STREAM source: <pre>FROM STREAM(<catalog>.<schema>.raw_customer_orders)</pre>
5	Create the gold SQL files	Create gold_customer_summary.sql and gold_region_summary.sql. Paste from the Lab 1 reference. These reference silver tables by short name — no catalog/schema prefix needed. SDP resolves them within the pipeline.

6	Confirm the bundle's catalog/schema match	Open databricks.yml and confirm targets.dev.variables.catalog and schema match the catalog/schema you used in the bootstrap notebook. If you picked something other than demos, update the catalog variable here too.
---	---	---

Tip — Want to skip hand-editing FROM clauses in every silver file? Use pipeline-level parameters (configurations) in the pipeline.yml and reference them in SQL via \${...}. Out of scope for this lab — for now, hard-code the catalog.schema literals to match your bootstrap.

Part 4 — Validate, Summary, Deploy

All commands run from inside the ball_test/ directory.

#	Action	Details
1	Validate	<pre>databricks bundle validate --target dev</pre> Parses every YAML, resolves variables, and reports schema or reference errors. Fast. Always run this first; fix any errors before continuing.
2	Summary	<pre>databricks bundle summary --target dev</pre> Prints the fully-resolved bundle: every resource (pipeline, job) with its final deployed name, catalog, schema, and workspace path. Use this to sanity-check that your interpolated values look right before you push to the workspace.
3	Deploy	<pre>databricks bundle deploy --target dev</pre> Uploads source files and creates/updates the pipeline and job in your workspace. Output ends with a tree of uploaded files and URLs of the created resources. With DATABRICKS_BUNDLE_ENGINE=direct exported (from the shell setup at the top), this should complete in under a minute.

If deploy fails with a signature or PGP error — Confirm DATABRICKS_BUNDLE_ENGINE=direct is exported in your current shell (echo \$DATABRICKS_BUNDLE_ENGINE). The bundled Terraform provider key has expired in some environments; the direct engine bypasses it.

Part 5 — Run the Pipeline

#	Action	Details
1	Trigger a full pipeline run	<pre>databricks bundle run ball_test_etl --target dev</pre> The CLI streams pipeline events: one entry per node (silver_customers, silver_orders, gold_customer_summary, gold_region_summary) plus constraint violation counts. Open the URL the CLI prints to follow along in the UI.
2	Verify outputs	In Catalog Explorer, browse to <catalog>.<schema>. You should see all four pipeline tables (silver_customers, silver_orders, gold_customer_summary, gold_region_summary) alongside the two bronze tables from the bootstrap. Row counts should match the Lab 1 expected outcomes: silver_customers = 10, silver_orders = 26.
3	Re-run a single transformation (optional)	<pre>databricks bundle run ball_test_etl --select silver_customers --target dev</pre> Lets you iterate on one node without re-running the whole DAG — handy while debugging a constraint or query.

Cleanup

When the workshop is over, tear down the deployed resources:

```
databricks bundle destroy --target dev
```

This removes the deployed pipeline, job, and workspace files. The bronze tables created by the bootstrap notebook are not part of the bundle — drop those manually if you want a clean slate:

```
DROP TABLE IF EXISTS <catalog>.<schema>.dim_customers_raw;  
DROP TABLE IF EXISTS <catalog>.<schema>.raw_customer_orders;
```

CLI Quick Reference

#	Action	Details
auth	Authenticate against a workspace	<code>databricks auth login --host https://<workspace>.cloud.databricks.com</code>
init	Scaffold a new bundle	<code>databricks bundle init</code>
validate	Check YAML + references	<code>databricks bundle validate --target dev</code>
summary	Preview resolved resources	<code>databricks bundle summary --target dev</code>
deploy	Push to workspace	<code>databricks bundle deploy --target dev</code>
run	Trigger the pipeline	<code>databricks bundle run ball_test_etl --target dev</code>
destroy	Tear down	<code>databricks bundle destroy --target dev</code>